

LABWORK 5

This document contains 10 real-life themed programming questions focused on working with arrays and functions in C.

Function 1: find_min_steps

- **Return Type:** `int`
 - **Parameters:** `int steps[7]`
 - **Description:** A health app stores the user's step counts for 7 days. Write a function that finds and returns the minimum step count (the lowest number of steps among all days).
 - **Example:**
 - Input: {4500, 5000, 3200, 8000, 6000, 7000, 4000}
 - Output: 3200
-

Function 2: reverse_array

- **Return Type:** `void`
 - **Parameters:** `int arr[], int size`
 - **Description:** A weather station stores weekly temperature data. Write a function that reverses the order of elements in the array and prints the result.
 - **Example:**
 - Input: {16, 18, 20, 19, 21, 22, 23}
 - Output: {23, 22, 21, 19, 20, 18, 16}
-

Function 3: find_first_occurrence

- **Return Type:** `int`
 - **Parameters:** `int arr[], int size, int target`
 - **Description:** An e-commerce system stores product IDs. Write a function that returns the index of the first occurrence of a given product ID. If the ID is not found, return -1.
 - **Example:**
 - Input: {110, 225, 340, 450, 225, 510}, target = 225
 - Output: 1
-

Function 4: calculate_average_and_above

- **Return Type:** `int`
- **Parameters:** `float expenses[], int size`
- **Description:** A user tracks their daily expenses for 10 days. Write a function that calculates the average expense, and another that returns the number of days above average.
- **Example:**
 - Input: {50, 60, 55, 80, 40, 70, 65, 90, 30, 50}

- Output: Average = 59, Days above average = 5

Function 5: count_occurrences

- **Return Type:** `int`
- **Parameters:** `int arr[], int size, int value`
- **Description:** A factory monitors the number of machine failures per day. Write a function that counts how many times a user-specified failure count appears in the array.
- **Example:**
 - Input: {2, 3, 2, 4, 2, 5}, value = 2
 - Output: 3

Function 6: update_grade

- **Return Type:** `void`
- **Parameters:** `int grades[], int index, int new_value`
- **Description:** A teacher wants to update a student's grade stored in an array. Write a function that replaces the grade at a given index with a new value. (Hint: The array should be passed by reference, not by value.)
- **Example:**
 - Input: {70, 80, 90}, index = 1, new_value = 85
 - Output: {70, 85, 90}

Function 7: filter_high_usage

- **Return Type:** `void`
- **Parameters:** `float usage[], int size, float result[], int *new_size`
- **Description:** A home tracks daily electricity usage for 30 days. Write a function that copies days with above-average usage into another array. Update *new_size to show the number of copied elements. Do not use dynamic memory allocation.
- **Example:**
 - Input: {5.2, 6.0, 4.8, 7.5, 6.3}
 - Output: result = {6.0, 7.5, 6.3}, new_size = 3

Function 8: detect_streak

- **Return Type:** `int` **Parameters:** `int scores[], int size`
 - **Description:** A company measures employee productivity daily. Write a function that detects if there is any 3-day increasing streak (scores increasing 3 days in a row). Return 1 if such a streak exists, otherwise return 0.
 - **Example:**
 - Input: {3, 4, 5, 2, 6, 7, 8}
 - Output: 1
-

Function 9: arrays_equal

- **Return Type:** `int`
 - **Parameters:** `int arr1[], int arr2[], int size`
 - **Description:** A delivery company compares two arrays containing daily delivery counts from two trucks. Write a function that checks if both arrays are identical. Return 1 if they are the same, 0 otherwise.
 - **Example:**
 - Input:
 - `arr1 = {10, 20, 30}`
 - `arr2 = {10, 20, 30}`
 - Output: 1
-

Function 10: clean_invalid_data

- **Return Type:** `int`
 - **Parameters:** `int data[], int size`
 - **Description:** A sensor records 100 readings, where invalid measurements are marked as -999. Write a function that removes invalid entries and shifts the remaining data left. Return the new size of the valid dataset.
 - **Example:**
 - Input: `{10, -999, 20, 30, -999, 40}`
 - Output: `{10, 20, 30, 40}`, `new_size = 4`
-

Main Function (15 pts)

- **Function Name:** `main`
 - **Return Type:** `int`
 - **Parameters:** `none`
 - **Description:**
 - Ask the user for inputs to fill arrays and test the above functions.
 - Print results clearly and formatted.
 - Demonstrate function calling, passing arrays, and pointer behavior.
-

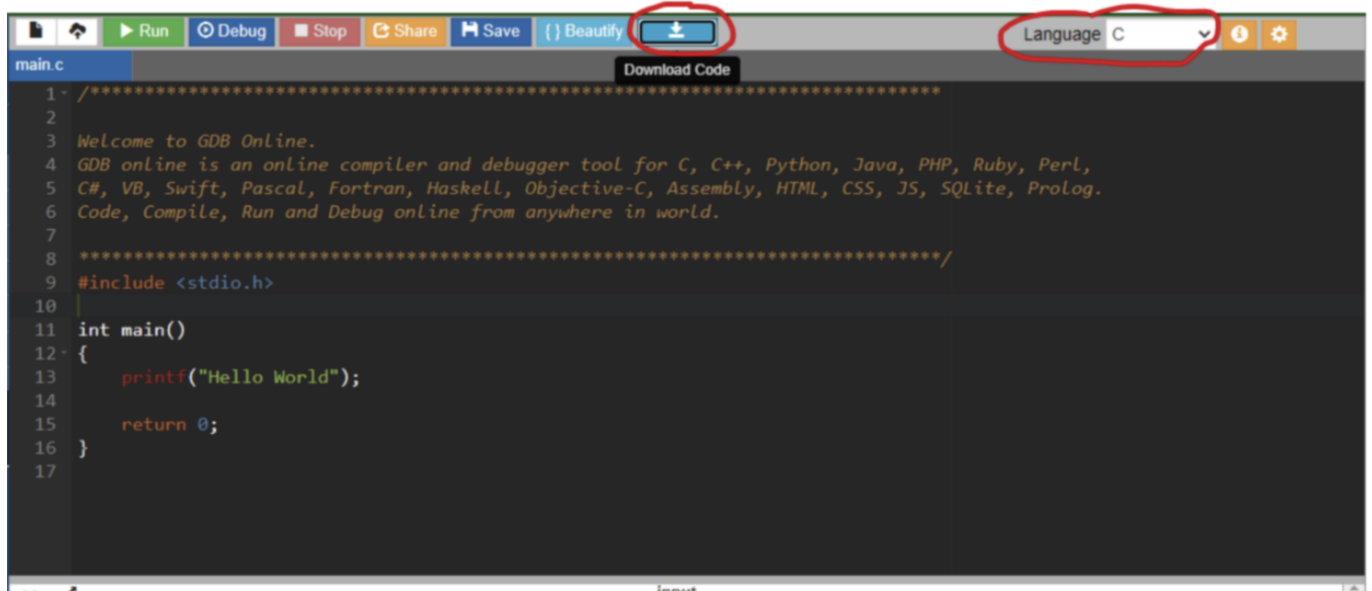
Submission Format

- Submit your `.c` file named as:
`name_surname.c`
- Late submissions will not be accepted.
- Follow function naming and signature exactly.
- Clear screen and print statements are optional, not graded.

Restrictions

- If you are not sure something is free to use or not, please ask your assistant.

- You have to complete the labwork using the functions provided. If you complete the labwork without using functions, your work will not be graded.
- Mobile phone and internet usage are not allowed.
- You can only access yulearn and the online C compiler <https://www.onlinegdb.com/> Do not forget to select the language if you use onlinegdb. When you finish your work, you can download your code by using the download code button on top of the window.



The screenshot shows the onlinegdb.com web interface. At the top, there is a toolbar with buttons for Run, Debug, Stop, Share, Save, Beautify, and a Download Code button (circled in red). To the right of the toolbar is a language selector dropdown menu (also circled in red) currently set to 'C'. Below the toolbar, the code editor displays a C program. The code includes a welcome message and a 'Hello World' program. A 'Download Code' button is visible above the code editor.

```
1  /*****
2
3  Welcome to GDB Online.
4  GDB online is an online compiler and debugger tool for C, C++, Python, Java, PHP, Ruby, Perl,
5  C#, VB, Swift, Pascal, Fortran, Haskell, Objective-C, Assembly, HTML, CSS, JS, SQLite, Prolog.
6  Code, Compile, Run and Debug online from anywhere in world.
7
8  *****/
9  #include <stdio.h>
10
11 int main()
12 {
13     printf("Hello World");
14
15     return 0;
16 }
17
```